

7.2 Nhạy cảm với Xoay trục tọa độ

- Hình Trái: Dữ liệu phân tách tuyệt đẹp bằng 1 đường thẳng ngang. (Cây sâu 1 tầng).
- Hình Phải: Cùng bộ dữ liệu nhưng xoay 45 độ. Cây buộc phải chia cắt thành vô vàn đường cầu thang ziczac dốc để xấp xỉ một đường chéo.
- Cây rườm rà, tổng quát hóa kém.

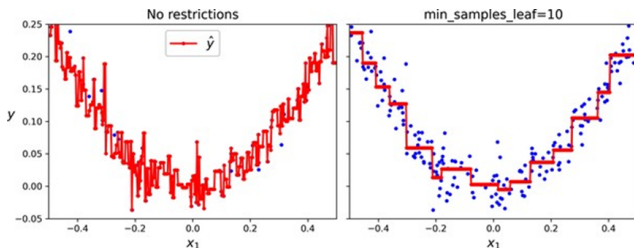


Figure: Hình 6-6: Độ nhạy cảm với trục xoay dữ liệu

7.3 Khắc phục Bằng PCA (Phân tích Thành phần Chính)

- Để giảm bớt sự nhạy cảm, ta có thể kết hợp Tiền xử lý bằng PCA (Principal Component Analysis).
- PCA sẽ xoay tập dữ liệu về trạng thái mà phương sai là lớn nhất trên các trục tọa độ.
- Nhờ đó, giới hạn khả năng cây bị bóp méo đường cầu thang.

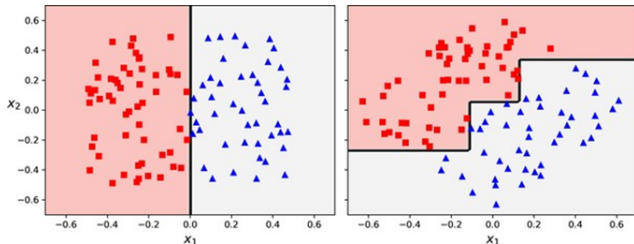


Figure: Hình 6-7: Cây phân loại sau khi xoay bằng PCA

7.4 Phương sai cao (High Variance)

- Vấn đề lớn nhất của Cây Quyết định: **Tính không ổn định cực cao (High Variance)**.
- Vì là một thuật toán tham lam (không kiểm tra lại toàn bộ tương lai), một **thay đổi vô cùng nhỏ** trong tập huấn luyện có thể tạo ra một cây **hoàn toàn khác biệt**.
- Một nhiễu loạn ngẫu nhiên nhỏ xíu ở nút gốc sẽ làm toàn bộ các nút con thay đổi.

7.5 Ví dụ về High Variance

- Trong bộ dữ liệu Iris, giả sử ta xóa **đúng 1 bông hoa** Versicolor to nhất (Dài 4.8cm, rộng 1.8cm).
- Cây mới học được (Bên phải) đã chia cắt dữ liệu theo ranh giới khác hoàn toàn so với cây ban đầu (Hình 6-2).

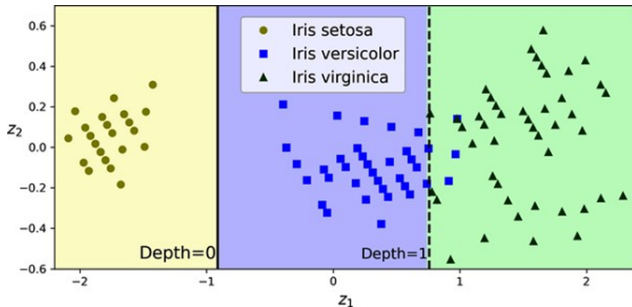


Figure: Hình 6-8: Cây bị méo mó chỉ sau khi xóa 1 điểm

7.6 Ngẫu nhiên tính trong Huấn luyện

- Ngay cả khi bạn **không xóa** bất kỳ dữ liệu nào, bản thân Scikit-Learn cũng chọn đặc trưng theo cách **ngẫu nhiên** để tăng tốc độ phân chia.
- Nghĩa là, huấn luyện cùng 1 tập dữ liệu 2 lần có thể ra 2 kết quả khác nhau (Nếu không set `random_state=42`).
- Sự ngẫu nhiên và Phương sai cao này thoạt nghe có vẻ là điểm yếu chí mạng...

7.7 Từ Điểm yếu trở thành Siêu Vị Thần (Random Forest)

- Thay vì dùng 1 cái cây không ổn định, ta sẽ trồng hàng nghìn cái cây như thế!
- Bằng cách trung bình hóa dự đoán của một **Khu Rừng (Forest)** các cây có phương sai cao và độc lập nhau.
- Kết quả là thuật toán **Random Forest** - một trong những thuật toán ML vĩ đại nhất lịch sử ra đời!

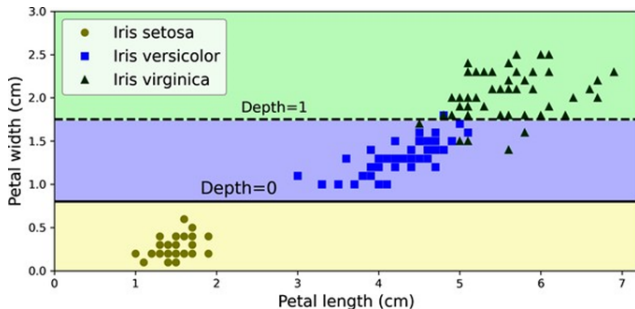


Figure: Hình 6-9: Khu rừng Random Forest (Chương 7)

8.1 Ưu điểm tóm tắt

- **Dễ hiểu, Dễ giải thích:** Có thể xuất ra dạng quy tắc (If-Else) hoặc sơ đồ luồng.
- **Không yêu cầu Chuẩn hóa:** Hoạt động tốt ngay cả khi các biến khác đơn vị, hoặc lẫn lộn chữ và số.
- **Xử lý Phi tuyến tính:** Cắt xẻ không gian tọa độ cực tốt, khớp với ranh giới phức tạp.
- **Nhanh ở khâu Dự đoán:** Suy luận với tốc độ Logarit $O(\log_2 m)$.

8.2 Nhược điểm tóm tắt

- **Overfitting nặng nề:** Thích ôm chặt lấy dữ liệu huấn luyện nếu không bị kìm hãm bằng Regularization.
- **Nhạy cảm với Xoay trục:** Bị trói buộc vào các ranh giới song song trục tọa độ (Khắc phục bằng PCA).
- **Instability (Bất ổn / Variance Cao):** Rung lắc mạnh khi dữ liệu chỉ đổi một tẹo. (Khắc phục bằng Ensemble Learning).

8.3 Bài tập 1: Phân loại Moons

- Khởi tạo bộ dữ liệu 10.000 điểm bằng `make_moons(n_samples=10000, noise=0.4)`.
- Phân chia thành Train/Test bằng `train_test_split`.
- Dùng `GridSearchCV` với `DecisionTreeClassifier` để dò tìm siêu tham số `max_leaf_nodes` tốt nhất.
- Huấn luyện và tính Accuracy Score.

8.4 Bài tập 2: Tự xây dựng Khu rừng (Forest)

- Bổ sung từ Bài tập 1. Thay vì 1 cây, hãy tạo 1000 tập con dữ liệu từ Train set.
- Dùng vòng lặp huấn luyện 1000 Cây Quyết Định (với cấu hình tốt nhất từ Bài 1) trên mỗi tập con.
- Dự đoán trên Test set: Bằng cách áp dụng **Mode** (Lấy dự đoán được số đông vote nhiều nhất) từ 1000 cây cho mỗi mẫu.
- Độ chính xác của bạn sẽ tăng hơn ít nhất 0.5 – 1.5% so với 1 cây.

HỎI & ĐÁP